Rajarshi Goswami

cs24mtech12012@iith.ac.in | rajarshi.goswami13@gmail.com | +91 8753011618 | LinkedIn | GitHub

EDUCATION

Institution	Degree / Board	Passing Year	Score / CGPA
Indian Institute of Technology Hyderabad	B.Tech	2027	8.16
Gurukul Grammar School	Class XII (CBSE)	2014	92.00%
Modern High School	Class X (CBSE)	2012	9.60%

RELEVANT COURSES

Topics in Compiler Optimization

Topics in Deep Learning

Software Verification

Topics in Neurosymbolic Programming

EXPERIENCE

Machine Learning Engineer (NLP) -- Patdel Analytics (Promax Legal Solution Pvt. Ltd.), Mohali

- Built an **XLNet multi-label classifier** for invention disclosures across **2,000+ patent categories**; resolved training collapse from extreme label sparsity by down-weighting negative-class loss terms, achieving **90% accuracy** and **cutting manual search time by 80%**.
- Developed an unsupervised **BERT Semantic Text Similarity** model for automated patent invalidity searches, successfully retrieving target reference patents within the **top-20 results for 90% of candidate cases**.
- Architected a context-aware synonym recommendation **API service on AWS ECS** for a freedom to operate (FTO) search tool, interfacing with a **SageMaker-hosted BERT MLM model** to predict domain-specific terms across extracted noun phrases in the input text.
- Resolved an **AWS SageMaker production latency regression** (slashing query times from **2 minutes to under 5s**) by replacing the default Hugging Face MLM pipeline with a **custom batched inference loop**.

RESEARCH EXPERIENCE

Elixir: A neurosymbolic Framework for Automated program specification synthesis

- Designed a neurosymbolic RL pipeline combining a **Graph Neural Network (GNN)** encoder and a **grammar-constrained decoder** to synthesize formal program specifications.
- **Integrated** a **Z3 backed verifier** for symbolic verification, leveraging **Counterexample-Guided Abstraction Refinement (CEGAR)** and novel behavioral rewards to address the **sparse-reward problem** with **partial-correctness feedback**.
- Accepted for presentation at the **International Workshop on Synthesis (SYNT) 2026**, Lisbon, Portugal.

Research Scholar

- Deep learning for substance-dependence detection from EEG signals and biomedical images

PUBLICATIONS

R. Goswami and A. Mishra, *Liquify Your Programs*. 15th International Workshop on Synthesis (SYNT), 2026. [Link]

PROJECTS

Realtime Event-Driven Conversational Platform

- Developed a realtime conversational platform integrating DialoGPT model with an **Apache Kafka event-streaming architecture.**
- Configured **Kafka Zookeeper and Broker** servers to manage asynchronous message routing and handle real-time text preprocessing.
- Developed a **full-stack web application** utilizing WebSockets for bidirectional communication, containerizing with **Docker** behind an **Nginx reverse proxy.**
- Tools: *Apache Kafka* |*Transformers* | *Flask* | *JavaScript* | *Docker* | *Nginx*

Custom YOLO Object Detection Pipeline

- Constructed a custom **YOLO object detection head** integrated with a **ResNet50 CNN backbone**, executing end-to-end model training, loss optimization, and evaluation.
- Formulated an automated data preprocessing and image augmentation pipeline.
- Deployed the detection model as a **containerized REST service on Microsoft Azure** using **Docker and Flask.**
- Tools: *Tensorflow* | *Pandas* | *Seaborn* | *Albumentations* | *Flask* | *Docker*

Image similarity based E-Commerce recommendation Engine

- Architected an unsupervised visual recommendation engine, implementing **fast vector search** via **Spotify Annoy (ANN indexing)** across a large Amazon product database.
- Designed an image feature extraction pipeline using a **VGG16 CNN backbone** to generate high-dimensional visual embeddings for context-aware product matching.
- Deployed an interactive web application enabling image upload and URL search, integrating **Microsoft Azure** for scalable asset retrieval.
- Tools: *TensorFlow* | *Spotify Annoy* |*Streamlit* | *Microsoft Azure*

Multi-Label Toxic Comment Classification Framework

- Constructed a custom **Bidirectional LSTM model from scratch in PyTorch** to execute multi-label toxicity classification.
- Engineered an end to end NLP preprocessing pipeline using **Torchtext** and **Pandas** for tokenization, vocabulary building, and sequence vectorization.
- Deployed a containerized web application on **Microsoft Azure**, serving the **Flask** app via an **Apache HTTP Server** in a **Docker container.**
- Tools: *PyTorch* | *Torchtext* | *Flask* | *Docker* | *Apache* | *Microsoft Azure*

Business Entity Resolution at Scale: Amazon ML Challenge 2026

- Built an out-of-core data pipeline matching 2.2M businesses against 10.3M records on a single 32GB machine, achieving a **0.934 macro F0.5 score**.
- Designed **blocking and candidate generation** for one to many entity matching, where each business has an unknown, variable number of true matches, across 336M candidate pairs.
- Extracted a custom **transliteration** dictionary directly from labeled training data to **bridge Latin and Indic-script names** without relying on external APIs or pretrained models.
- Tools: *DuckDB* | *LightGBM* | *RapidFuzz*

Compiler for Simply Typed Lambda Calculus with LLVM Backend

- Architected lambdac, an end-to-end **compiler frontend and intermediate pipeline** in C++20, using **zero-copy text processing** and a **cache-optimized lookup** strategy during lexing to minimize dynamic allocation overhead and cache misses in syntax analysis.
- Enforced strict compile-time exhaustiveness checking across all AST transformations, with a custom **arena allocation strategy** guaranteeing **pointer stability** through alpha-renaming and scope transformations.
- Designed a custom IR-level optimization pass using **escape analysis** to promote non-escaping **heap allocations to stack memory**, eliminating allocator overhead and bookkeeping latency for eligible closures and achieving a **1.88x JIT execution speedup**, verified through a controlled, correctness-checked benchmark.
- Built an in-process JIT execution path for dynamic code generation, calling compiled machine code directly via native function pointers to **eliminate per-execution process-spawn overhead** inherent to subprocess-based execution.
- Tools**:** *C++* | *LLVM*

SKILLS

- **Languages:** Python, C, C++, OCaml, Linux/UNIX Shell
- **ML Libraries:** NumPy, Pandas, PyTorch, Tensorflow, Scikit-Learn, NLTK, Transformers
- **Deployment:** Amazon Web Services (AWS), Microsoft Azure, Docker
- **System Frameworks:** LLVM